

A Technical Introduction to Batch normalization

$$\hat{X}_i = \frac{x_i - \mu_B}{\sqrt{\sigma_B^2 + \varepsilon}}$$

$$y_i = \gamma \hat{X}_i + \beta$$

Section 1

$\min_{w \in \mathbb{R}^d} \rho(w) = \min_{w \in \mathbb{R}^d} (-w^T u + \frac{1}{2} \|w\|_2^2)$ where $\rho(w) = \min_{w \in \mathbb{R}^d} \frac{1}{2} \|w\|_2^2 - w^T u$. Note that this objective is a form of the generalized Rayleigh quotient

Section 2

$s_t = s(w_t, \gamma_t) = -\|w_t\|_2 \log(w_t, \gamma_t)$ where $s_t = -\frac{1}{2} \log \frac{\|w_t\|_2^2}{\|w_t\|_2^2 + \gamma_t^2}$. For each step, if $h(w_t, \gamma_t) \neq 0$, then update the direction as

Section 3

$\rho(w_t) - \rho(w^*) \leq (1 - \mu_L) 2^t (\rho(w_0) - \rho(w^*))$ where $\rho(w) = \frac{1}{2} \|w\|_2^2 - w^T u$.

Section 4

$h(w_t, \gamma_t) = E_z [\phi'(z^T w_t)] (u^T w_t) - E_z [\phi''(z^T w_t)] (u^T w_t)^2$ where $\phi(z) = E_z [\phi(z^T w_t)]$. Let the step size be

Section 5

$\|w_t - f(w_t)\|_2 \leq (1 - \mu_L) 2^t \Phi(2(\rho(w_0) - \rho^*) + 2 - T \zeta |b_t(0) - a_t(0)| \mu^2)$ where $\Phi(x) = 1 - \frac{\mu}{2} \frac{1}{\|x\|_2} \frac{1}{\Phi(x)}$ and $\mu = \frac{1}{2} \frac{1}{\|u\|_2} \frac{1}{\Phi(u)}$, such that the algorithm is guaranteed to converge linearly. Although the proof stands on the assumption of Gaussian input, it is also shown in experiments that GDNP could accelerate optimization without this constraint.

Section 6

$\|w_t - f(w_t)\|_2 \leq (1 - \mu_L) 2^t C(\rho(w_0) - \rho^*) + 2 - T \zeta |b_t(0) - a_t(0)| \mu^2$ where $C(x) = 1 - \frac{\mu}{2} \frac{1}{\|x\|_2} \frac{1}{C(x)}$

$\|w_t - f(w_t)\|_2 \leq (1 - \frac{\mu}{2}) 2^t C(\rho(w_0) - \rho^*) + \frac{1}{2} \frac{1}{\|u\|_2} \frac{1}{C(u)}$. Since the parameters of each hidden unit converge linearly, the whole optimization problem has a linear rate of convergence.

Section 7

Neural networks Consider a multilayer perceptron (MLP) with one hidden layer and m hidden units with mapping from input $x \in \mathbb{R}^d$ to a scalar output described as

Section 8

$\gamma_t = \text{Bisection}(T_s, f, w_t)$ where $\text{Bisection}()$ is the classical bisection algorithm, and T_s is the total iterations ran in the bisection step. Denote the total number of iterations as T_d , then the final output of GDNP is

Section 9

ϵ is added in the denominator for numerical stability and is an arbitrarily small positive constant. The resulting normalized activation $\hat{x}^{(k)}$ have zero mean and unit variance, if ϵ is not taken into account. To restore the representation power of the network, a transformation step then follows as

Section 10

$\hat{g}_j \leq \gamma 2 \sigma_j^2 (g_j^2 - m \mu_j^2 - \lambda^2 \mu_j^2)$ where $\hat{g}_j = \frac{1}{\sigma_j^2} (g_j^2 - m \mu_j^2 - \lambda^2 \mu_j^2)$, where $g_j = m \max_{|X| \leq \lambda} |W_L|$ and $\hat{g}_j = m \max_{|X| \leq \lambda} |W_L|$. In addition to the smoother landscape, it is further shown that batch normalization could result in a better initialization with the following inequality:

Section 11

Vanishing/exploding gradients Even though batch norm was originally introduced to alleviate gradient vanishing or explosion problems, a deep batch norm network in fact suffers from gradient explosion at initialization time, no matter what it uses for nonlinearity. Thus, the optimization landscape is very far from smooth for a randomly initialized, deep batch norm network. More precisely, if the network has L layers, then the gradient of the first layer weights has norm $> c \lambda L$

λ for some $\lambda > 1$, $c > 0$ depending only on the nonlinearity. For any fixed nonlinearity, λ decreases as the batch size increases. For example, for ReLU, λ decreases to $\pi / (\pi - 1) \approx 1.467$ as the batch size tends to infinity. Practically, this means deep batch norm networks are untrainable. This is only relieved by skip connections in the fashion of residual networks. This gradient explosion on the surface contradicts the smoothness property explained in the previous section, but in fact they are consistent. The previous section studies the effect of inserting a single batch norm in a network, while the gradient explosion depends on stacking batch norms typical of modern deep neural networks.

Section 12

$$f_{\text{BN}}(w, \gamma, \beta) = E_{x \sim \phi(\text{BN}(x|T, w))} = E_{x \sim \phi(\gamma(x|T, w - E_{x \sim [x|T, w]} \vee \text{var}_{x \sim [x|T, w]}^{1/2}) + \beta)}$$

$$f_{\text{BN}}(w, \gamma, \beta) = E_{x \sim \phi(\text{BN}(x^T|w))} = E_{x \sim \phi(\gamma(x^T|w - E_{x \sim [x^T|w]} \vee \text{var}_{x \sim [x^T|w]}^{1/2}) + \beta)}$$

, by definition. The variance term can be simplified such that $\text{var}_{x \sim [x|T, w]} = w^T S w$. Assume that x has zero mean and β can be omitted, then it follows that